

Contents

Code Review: GPU Implementation Plan v1	1
Review Scope	1
Findings	1
Summary	18
Unreviewed Areas	18

Code Review: GPU Implementation Plan v1

Date: 2026-05-16 Reviewer: code-review agent (adversarial review of the plan document) Round: 1 (initial)

Review Scope

- Plan reviewed: `gpu_implementation_plan_v1.md`
- Predecessor analysis: `gpu_porting_analysis_2026-05-16.md`
- Files cross-checked:
 - `/Users/chunhuizhao/projects/seas-mfem-paraview/miniapps/seas/drivers/{seas,tpv102,tpv104}`
 - `/Users/chunhuizhao/projects/seas-mfem-paraview/miniapps/seas/common/mpi_context.hpp`
 - `/Users/chunhuizhao/projects/seas-mfem-paraview/miniapps/seas/dynamic/wave_operator.{hpp,}`
 - `/Users/chunhuizhao/projects/seas-mfem-paraview/miniapps/seas/friction/state_evolution.hp`
 - `/Users/chunhuizhao/projects/seas-mfem-paraview/miniapps/seas/fault/rate_state_fault.hpp`
 - `/Users/chunhuizhao/projects/seas-mfem-paraview/miniapps/seas/domain/elasticity_operator_`
 - MFEM: `general/device.{hpp,cpp}`, `general/forall.hpp`, `general/backends.hpp`,
`linalg/hypre.{hpp,cpp}`, `linalg/vector.cpp`, `mesh/mesh.{hpp,cpp}`
- Domain context consulted: `CLAUDE.md` (project policy on no silent fallbacks), `CODEBASE_GUIDE.md` (Q layout, ADER, fault basis), `MFEM_CHANGELOG`.

Findings

[R-GPU-001] CRITICAL [Phase 2 / friction kernels] — Virtual **StateEvolution::Rate** cannot be invoked from a device kernel

Category: BUG / ASSUMPTION

Description: Phase 2 instructs the implementer to tag `StateEvolution::Rate(V, psi, Dc)` with `MFEM_HOST_DEVICE` and call it from inside `mfem::forall`. But `state_evolution.hpp:41` shows `Rate` is a pure virtual function:

```
class StateEvolution {
    virtual real_t Rate(real_t V, real_t theta, real_t Dc) const = 0;
    // ... and AgingLawPsi/SlipLawPsi override it via "override" not by
    // re-declaration.
};
```

Tagging a virtual with `MFEM_HOST_DEVICE` does not make virtual dispatch work on the device — the vtable is constructed on the host and the pointer in the host-side derived-class instance is a host pointer. `evolution_>Rate(V_abs, psi, Dc)` in a `forall` body will segfault on NVIDIA hardware and produce wrong results on AMD.

Trigger: Any device build that runs `RateStateFaultOperator::ComputeRHSDevice` with the proposed plan, on a non-trivial fault DOF set.

Actual behavior: - CUDA: kernel SIGSEGV at `evolution_>Rate` dereference of the host vtable. - HIP: depending on memory mapping, either kernel hang or silent wrong values.

Expected behavior: The rate formula must be inlined in the device kernel, with the derived-class identity selected before kernel launch (e.g. an enum class EvolutionKind { AgingPsi, SlipPsi } member on RateStateFaultOperator, switched on inside the kernel).

Suggested fix (apply to Phase 2 plan body):

```
- Add `void ComputeRHSDevice(...)` implementation that:
-   - Launches `mfem::forall(num_nodes_, ...)` with the per-DOF loop
-     body inlined (no virtual calls to `evolution_>Rate`; the rate
-     formula must be coded inline as `MFEM_HOST_DEVICE` in
-     `friction/state_evolution.hpp`).
+ Add `void ComputeRHSDevice(...)` implementation that:
+   - At first call, reads the runtime-known evolution-law identity
+     from `evolution_>GetName()` and caches an `EvolutionKind`
+     enum (`AgingPsi`, `SlipPsi`, `Aging`, `Slip`). Inlining the
+     polymorphic dispatch as a switch inside the kernel body –
+     virtual calls are not legal in device code.
+   - Launches `mfem::forall(num_nodes_, ...)` with the rate formulas
+     coded as `MFEM_HOST_DEVICE` *free functions* in
+     `friction/state_evolution_kernels.hpp` (new file). Each
+     free function computes the rate for one law identity and is
+     selected by the cached enum via a `switch` inside the kernel.
+   - Drop the proposal to tag the virtual `Rate` method itself with
+     MFEM_HOST_DEVICE – that has no effect on device-side dispatch.
```

Additionally, the Phase 2 “Files to Modify” list must add: - miniapps/seas/friction/state_evolution_kernels.hpp (NEW; defines MFEM_HOST_DEVICE free functions AgingLawPsiRateDevice(V, psi, Dc, b, V0, f0), etc.). - miniapps/seas/fault/rate_state_fault.hpp — add an evolution_kind_ enum member populated in the constructor from evolution_>GetName(), consumed by ComputeRHSDevice.

Test case:

```
TEST(R_GPU_001_FrictionDevice, NoVirtualCallOnDevice) {
    // Setup AgingLawPsi
    AgingLawPsi aging(/*b=*/0.012, /*V0=*/1e-6, /*f0=*/0.6);
    FaultGeometry geom = ...;
    RateStateFaultOperator<ParMesh, 2> op(&geom, ..., &aging, ...);
    Device device("cuda");
    Vector traction(2 * N), state(3 * N), rate(3 * N), nt(N);
    // (initialize with reasonable values)
    // This call MUST NOT segfault and MUST produce values matching the
    // CPU branch within 1e-14.
    op.ComputeRHS(traction, state, rate, &nt);
    Vector rate_cpu(3 * N);
    Device device_cpu("cpu");
    op.ComputeRHS(traction, state, rate_cpu, &nt);
    ASSERT_LT((rate - rate_cpu).Norml2(), 1e-14 * rate_cpu.Norml2());
}
```

[R-GPU-002] CRITICAL [Phase 0 / drivers] — TPV102/104/205 drivers do not use **MPIContext**; plan’s insertion point is wrong

Category: BUG / ASSUMPTION

Description: Phase 0 instructs the implementer to “Place the call [to seas::InitDevice] after MPIContext construction” in all four drivers. This is true for drivers/seas_driver.cpp (line 190) but false for the three

dynamic-rupture drivers: - `tpv102_driver.cpp:413-423` uses `raw MPI_Init(&argc, &argv)` and never instantiates `MPIContext`. - `tpv104_driver.cpp:412` — same. - `tpv205_driver.cpp:452` — same.

Worse, none of those three drivers call `Hypre::Init()` (they are explicit, no linear solve). The plan's stated init order `Mpi::Init` → `Hypre::Init` → `Device::Configure` therefore cannot be followed verbatim — `Hypre::Init` is never called.

An implementer following the plan literally will either: - Add `MPIContext` construction next to `MPI_Init` (changes semantics: `MPIContext` calls `Hypre::Init()` even for runs that don't need HYPRE, drags HYPRE into the explicit-solver link line); OR - Add `seas::InitDevice` next to `MPI_Init` and miss the `Hypre::Init` prerequisite (which is fine because Hypre isn't used, but the plan's contract is violated).

Trigger: Implementer reads Phase 0, applies pattern to `tpv102_driver.cpp`, finds no `MPIContext`, guesses what to do.

Expected behavior: The plan must distinguish: - BP5 driver (`seas_driver.cpp`) — uses `MPIContext` already (which calls `Mpi::Init` + `Hypre::Init`); insert `seas::InitDevice` after. - TPV102/104/205 drivers (`*_driver.cpp`) — use raw `MPI_Init`; insert `seas::InitDevice` directly after `MPI_Init`; do NOT call `Hypre::Init` (it is not needed for explicit drivers).

`Device::Configure` is safe when HYPRE is not initialized — it gates on `if (HYPRE_Initialized()) { Hypre::InitDevice(); }` (`general/device.cpp:287`).

Suggested fix (Phase 0 plan body):

```
- `miniapps/seas/drivers/seas_driver.cpp` - at the top of `main` (after
- `MPIContext mpi(&argc, &argv)` which already calls `Mpi::Init` +
- `Hypre::Init` per [common/mpi_context.hpp:36-47]...):
+ `miniapps/seas/drivers/seas_driver.cpp` - at the top of `main` (after
+ `MPIContext mpi(&argc, &argv)` which already calls `Mpi::Init` +
+ `Hypre::Init` per [common/mpi_context.hpp:36-47]...):
... existing snippet ...
+
+ `miniapps/seas/drivers/tpv102_driver.cpp`,
+ `miniapps/seas/drivers/tpv104_driver.cpp`,
+ `miniapps/seas/drivers/tpv205_driver.cpp` - these drivers use raw
+ `MPI_Init` (NOT `MPIContext`) and do NOT call `Hypre::Init`.
+ Insert `seas::InitDevice` directly after the `MPI_Init` /
+ `MPI_Comm_rank` block at lines 415-423. Do NOT add MPIContext
+ here - it would pull `Hypre::Init` into an explicit-solver build
+ for no reason. The `seas::InitDevice` call still works because
+ `Device::Configure` gates the Hypre-init step on
+ `HYPRE_Initialized()`.
- `miniapps/seas/drivers/tpv102_driver.cpp`, `tpv104_driver.cpp`,
- `tpv205_driver.cpp` - same insertion pattern. Place the call **after**
- `MPIContext` construction. Order is enforced by the constructor of
- `MPIContext` calling `Hypre::Init`; `InitDevice` (which calls
- `Device::Configure`) must follow.
```

Test case:

```
TEST(R_GPU_002_DriverInit, Tpv102DriverDoesNotRequireHypreInit) {
    // Build tpv102_driver with --device cuda and run a 1-step smoke.
    // ASSERT no abort from Device::Configure (it must gate on
    // HYPRE_Initialized()).
}
```

[R-GPU-003] CRITICAL [Phase 3 / mass inverse] — **MAX_DOF = 84** is wrong for p=4 tet (claim mis-states supported order)

Category: BUG / ASSUMPTION

Description: Phase 3's ApplyMassInverseDevice proposal uses `real_t rhs_local[MAX_DOF]` with `MAX_DOF = 84` (p=4 tet). For an L2 DG polynomial of order p on a tetrahedron, DOF count is $\binom{p+3}{3} = (p+1)(p+2)(p+3)/6$:

p	ndof on tet
0	1
1	4
2	10
3	20
4	35
5	56
6	84

So `MAX_DOF = 84` actually accommodates p=6, not p=4. While the oversizing is benign (allocates a larger stack array than strictly needed), the plan's "abort at p > 4" instruction is then inconsistent with the buffer it allocates. More dangerously, an implementer reading this might believe p=4 tet has 84 DOFs and write loops bounded by `MAX_DOF` instead of `ndof` — causing out-of-bounds reads of uninitialised stack memory.

Also: `tpv102_driver` defaults to `--order 1` (line 457) and BP5 TOML uses `order = 1` (`bp5_500m.toml:15`). Production rarely exceeds p=2, so the abort gate at p>4 is unlikely to trip — but the buffer-size claim is still wrong.

Trigger: Implementer mis-sizes `rhs_local` based on the wrong "`MAX_DOF = 84` for p=4 tet" comment, or accidentally writes a loop indexed by `MAX_DOF` rather than `ndof`.

Expected behavior: - Plan should clearly state `MAX_DOF_TET_P4 = 35` (or use the formula). - Stack array should be sized for the maximum supported order. - The abort gate `MFEM_VERIFY(ndof <= MAX_DOF, ...)` should match the buffer size used.

Suggested fix (Phase 3 plan body):

```
- - `MAX_DOF` = 84 (p=4 tet) - register-resident.
+ - `MAX_DOF_TET` is computed from the supported maximum order via
+   `(p+1)(p+2)(p+3)/6`. For supported orders  $p \in \{1,2,3,4\}$ ,
+   `MAX_DOF_TET = 35`. Extend to p=5 (56) or p=6 (84) only when
+   verified in tests. All loops MUST be bounded by the actual
+   `ndof` at runtime, never by `MAX_DOF_TET`.
- - For p > 4, fall back to shared-memory tiling (defer to Phase 7).
+ - For p > 4 (ndof > 35), fall back to shared-memory tiling (defer
+   to Phase 7). Phase 3 aborts with
+   `MFEM_VERIFY(ndof_per_el_ <= MAX_DOF_TET, ...)` at first
+   `InitDeviceBuffers()` call.
```

Also update the corresponding line in §"Edge Cases to Handle":

```
- **Element order > 4** - `MAX_DOF = 84` register array overflows.
+ **Element order > 4** - `MAX_DOF_TET = 35` stack array would overflow
+   if the kernel writes beyond `ndof`. Abort with
+   `MFEM_VERIFY(ndof_per_el_ <= 35, ...)` for p=4 tet.
```

Test case:

```

TEST(R_GPU_003_MassInverseDevice, P4TetDofCountIs35) {
    Mesh mesh = MakeSingleTetMesh();
    for (int p = 1; p <= 4; ++p) {
        L2_FECollection fec(p, 3, BasisType::GaussLobatto);
        FiniteElementSpace fes(&mesh, &fec);
        const int expected[] = {0, 4, 10, 20, 35};
        ASSERT_EQ(fes.GetFE(0)->GetDof(), expected[p]);
    }
}

```

[R-GPU-004] CRITICAL [Phase 2 / friction] — Proposed **AtomicMax** is not available in MFEM

Category: BUG / ASSUMPTION

Description: Phase 2's first proposal for the `V_max_` reduction reads: > B. Atomic-max inside the kernel into a Vector `v_max_(1)`.

But MFEM's `general/backends.hpp` only provides `AtomicAdd<T>` (line 94). There is no `AtomicMax` template — neither CUDA's `atomicMax` for double nor a wrapper exists. Implementer following option B would either: - Write raw `atomicMax(...)` — fails to compile on CUDA for double. - Write a `atomicCAS` loop — possible but error-prone and not in MFEM.

The plan correctly chooses option A (scratch Vector + `Vector::Max()`), but should call out that option B is not viable, not just “simpler”.

Trigger: Implementer chooses option B (atomic-max) reading the plan as neutral between A and B.

Expected behavior: Plan must mark option B as “NOT VIABLE — MFEM has no `AtomicMax`” and direct the implementer to option A unconditionally.

Suggested fix (Phase 2 plan body):

```

4. **V_max reduction.** Two acceptable patterns:
- **A.** Per-DOF kernel writes `V_abs` to a scratch `Vector v_abs_scratch_(num_nodes_)`, marked `UseDevice(true)`. Post-kernel host call to `v_abs_scratch_.Max()` (MFEM `Vector::Max()` runs on device when `UseDevice(true)`).
- **B.** Atomic-max inside the kernel into a `Vector v_max_(1)`.
+ **B.** NOT VIABLE — MFEM provides only `mfem::AtomicAdd<T>` in `general/backends.hpp`; there is no `AtomicMax` template and CUDA's `atomicMax` does not have a `double` overload. Do not pursue this path.
- Choose A (simpler, no atomics on `real_t`, well-tested MFEM path).
+ Choose A (the only available path).

```

Test case:

```

TEST(R_GPU_004_AtomicMax, NoAtomicMaxTemplate) {
    // This is a documentation test: confirm AtomicMax is not declared.
    // grep -q "AtomicMax" mfem/general/backends.hpp must be empty.
    // (Compile-time enforcement via static_assert is also acceptable.)
}

```

[R-GPU-005] CRITICAL [Phase 4 / face flux] — Plan says “atomic adds” but does not specify `mfem::AtomicAdd`; implementer may write non-portable CUDA `atomicAdd`

Category: BUG / ASSUMPTION

Description: Phase 4’s race-resolution Pattern A reads: > A. Atomic adds — `atomicAdd` on `rhs[c*ndof_total + e*ndof + i]`. On A100/MI250X, double-precision atomics are slow but available.

But raw `atomicAdd(double*, double)` requires `__CUDA_ARCH__ >= 600` (`general/backends.hpp:69–91` provides a CAS-loop fallback for older architectures, but it’s inside namespace `mfem`). On HIP, the symbol is `atomicAdd` too but in a different namespace. The portable way is `mfem::AtomicAdd<T>` (line 94 of `backends.hpp`), which dispatches correctly per backend.

If the implementer writes raw `atomicAdd(...)`, the HIP build will fail to compile (or worse, the CPU `--device cpu` path will compile against some host stub that doesn’t actually atomic, breaking OpenMP runs).

Trigger: Implementer reads “atomic adds” as a license to use raw `atomicAdd`.

Expected behavior: Plan must explicitly direct use of `mfem::AtomicAdd<real_t>(rhs[i], val)`.

Suggested fix (Phase 4 plan body):

```
- - **A.** **Atomic adds** – ‘atomicAdd’ on ‘rhs[c*ndof_total + e*ndof + i]’.
-   On A100/MI250X, double-precision atomics are slow but available.
+ - **A.** **Atomic adds via ‘mfem::AtomicAdd<real_t>’** (declared in
+   ‘general/backends.hpp:94’). Do NOT use raw ‘atomicAdd’ – only
+   ‘mfem::AtomicAdd<T>’ is portable across CUDA, HIP, and the OpenMP
+   host fallback. On compute capability ≥ 6.0 (A100/V100) and
+   MI200+, double atomics are fast enough; the CUDA backends.hpp
+   provides a CAS-loop fallback for older architectures.
```

Also add a kernel-coding-rule line in the Convention constraints section (top of the plan):

```
- Avoid ‘std::vector’, ‘std::map’, ‘std::sort’, virtual dispatch, exceptions.
- Treat ‘DenseMatrix::Mult’ etc. as forbidden – use raw scalar loops.
+ - For accumulation into shared output slots, use
+   ‘mfem::AtomicAdd<real_t>(slot, val)’ from ‘general/backends.hpp’.
+   Never use raw CUDA / HIP ‘atomicAdd’ (not portable).
```

Test case:

```
TEST(R_GPU_005_AtomicAddPortable, FaceFluxAtomicBuildsOnHipAndCuda) {
    // Build the face-flux kernel under both MFEM_USE_CUDA=YES and
    // MFEM_USE_HIP=YES; both must compile and link.
}
```

[R-GPU-006] MODERATE [Phase 0 / `seas::InitDevice`] — Claim that “MFEM permits only one `Device::Configure` call” is incorrect

Category: ASSUMPTION / QUALITY

Description: Phase 0’s Edge Cases section reads: > Repeated `Device::Configure` calls — MFEM permits only one. Make `InitDevice` guarded by a static `bool` called `= false` and warn (root rank only) if invoked twice with the same config; abort on conflicting configs.

This is wrong. Reading `general/device.cpp:191–293`, `Device::Configure` does not enforce single-call semantics. Internally:

```
Get().MarkBackend(it->second); // line 242: backends |= b (additive)
```

A second call with a different backend string just ORs in additional backends; no abort or warning. The early-return if (device_env) only applies when the MFEM_DEVICE env var is set.

The static-guard idea in `seas::InitDevice` is itself fine, but the justification in the plan is incorrect, and the abort criterion (“abort on conflicting configs”) is harder to implement than the plan suggests (you’d have to track which backends got marked).

Trigger: Implementer reads the plan and writes an overly strict guard that aborts on a legitimate re-init (e.g., a test that resets the device).

Expected behavior: Plan should say: `seas::InitDevice` is idempotent on identical inputs; on different inputs, log a warning (since MFEM will silently add backends rather than replace them); abort is optional and only strictly required if conflicting backends are passed in.

Suggested fix (Phase 0 plan body):

```
- **Repeated `Device::Configure` calls** – MFEM permits only one. Make
-   `InitDevice` guarded by a `static bool called = false` and warn
-   (root rank only) if invoked twice with the same config; abort on
-   conflicting configs.
+ **Repeated `Device::Configure` calls** – MFEM does NOT enforce
+   single-call semantics; `Device::Configure` is additive – a second
+   call with a different backend ORs that backend into the active
+   set (see `general/device.cpp:242`, `MarkBackend`).
+   Make `seas::InitDevice` guarded by a function-static
+   `bool initialized = false` plus a cached config string; the second
+   call is a no-op if the config matches, a warning if it differs but
+   strict-equality is not required (e.g., tests may re-init). Do
+   not attempt to “abort on conflicting configs” – there is no
+   reliable cross-call comparison.
```

Test case:

```
TEST(R_GPU_006_InitDeviceIdempotent, SecondCallNoOp) {
    seas::InitDevice("cpu");
    seas::InitDevice("cpu");           // must not abort, must not warn
    ASSERT_EQ(seas::GetActiveBackend(), "cpu");
}
TEST(R_GPU_006_InitDeviceIdempotent, SecondCallDifferentConfigWarns) {
    seas::InitDevice("cpu");
    testing::internal::CaptureStderr();
    seas::InitDevice("cuda");
    std::string out = testing::internal::GetCapturedStderr();
    ASSERT_THAT(out, HasSubstr("seas::InitDevice already configured"));
}
```

[R-GPU-007] MODERATE [Phase 1 / BP5 HYPRE] — “GPU vs CPU residual within 1 iteration” claim is unsupported

Category: ASSUMPTION

Description: Phase 1 lists a numerical constraint: > HYPRE PCG+AMG GPU vs CPU on identical matrix: residual reduction trajectory differs by ≤ 1 iteration count, final solution L2 relative diff $< 1e-10$.

Both halves are unsupported by data. BoomerAMG GPU uses `l1Jacobi` smoother by default (`linalg/hypre.cpp:1504, 18`) where the CPU uses `l1GS`; the two converge differently. Iteration counts can differ by 5–20× depending on the matrix conditioning, mesh aspect ratio, and `interp_type` — not “ ≤ 1 ”. And the final solution tolerance

is typically $< 1e-8$ (PCG convergence tol), not $< 1e-10$ — pushing AMG to $1e-10$ requires inner-iteration tightening that's a separate decision.

If the test author implements this acceptance criterion literally, the test will fail intermittently even on a correct GPU port.

Trigger: test_bp5_hypre_gpu runs on a representative BP5 stiffness matrix.

Expected behavior: Plan must use loose, realistic tolerances: - Iteration-count drift: “within $2\times$ ” is more realistic than “ ≤ 1 ”. - Final-solution L2 relative diff: $< 1e-6$ is the practical AMG cross-architecture tolerance; tighter requires CG tol $< 1e-12$ and is over-specifying.

Suggested fix (Phase 1 plan body):

```
- - HYPRE PCG+AMG GPU vs CPU on identical matrix: residual reduction
- trajectory differs by  $\leq 1$  iteration count, final solution L2 relative
- diff  $< 1e-10$ .
+ - HYPRE PCG+AMG GPU vs CPU on identical matrix: iteration counts may
+ differ by up to  $2\times$  (different AMG smoother — `l1Jacobi` on GPU vs
+ `l1GS` on CPU per `linalg/hypre.cpp`); final solution L2 relative
+ diff  $< 1e-6$  at PCG tolerance  $1e-8$ . Tighter solution tolerance
+ (e.g.,  $1e-10$ ) requires PCG tol  $1e-12$  and an explicit
+ `SetPrintLevel` audit — out of scope for Phase 1 acceptance.
```

Apply the same fix to Phase 1 acceptance criterion:

```
- [ ] `test_bp5_hypre_gpu` passes: same `K`, same `b`, GPU solution and
- CPU solution differ by  $< 1e-10$  relative L2 (HYPRE BoomerAMG is
- not bit-identical CPU/GPU but is convergent).
+ [ ] `test_bp5_hypre_gpu` passes: same `K`, same `b`, GPU and CPU
+ BoomerAMG-preconditioned PCG both converge within  $2\times$  iteration
+ count of each other, with L2 relative diff  $< 1e-6$  at PCG
+ tolerance  $1e-8$ .
```

Test case: (rewritten test acceptance — replace $< 1e-10$ literal with $< 1e-6$ and add iteration-count drift check $< 2 * \text{cpu_iters}$).

[R-GPU-008] MODERATE [Phase 3 / volume kernel] — **Q_data** referenced inside **forall** without **Read()** capture

Category: BUG / EDGE_CASE

Description: Phase 3's pseudocode for the volume kernel reads:

```
mfem::forall(ne, [=] MFEM_HOST_DEVICE (int e) {
    const real_t *Q_e = Q_data + e * ndof;    // ...
    for (int q = 0; q < nqp; q++) {
        // ...
        Q_qp[c] += B(i, q) * Q_data[c*ndof_total + e*ndof + i];
    }
}
```

Q_data must be obtained inside the host setup as `const real_t *Q_data = Q.Read();` (with `Q.UseDevice(true)` set). The pseudocode does not show the capture. An implementer copying the pseudocode might write `Q.GetData()` (host pointer) or omit the sync entirely, producing stale data on device.

Also, the pseudocode references `B(i, q)` and `G(e, i, d, q)` as multidimensional reshapes, but the surrounding text doesn't show how they were obtained from `qd.B.Read()` / `qd.G.Read()` and `Reshape(...)`. The MFEM convention is to do this in the host setup, not inside the kernel.

Trigger: Implementer copies pseudocode literally, misses the `Read()` calls.

Expected behavior: Pseudocode must show the host setup explicitly. Example:

```
const auto *Q_data = Q.Read();
auto *rhs_data = rhs.ReadWrite();
const auto B = Reshape(qd.B.Read(), ndof, nqp);
const auto G = Reshape(qd.G.Read(), ne, ndof, 3, nqp);
const auto wd = Reshape(qd.w_detJ.Read(), ne, nqp);
mfem::forall(ne, [=] MFEM_HOST_DEVICE (int e) { /* ... */ });
```

Suggested fix (Phase 3 plan body — replace the volume-kernel pseudocode block):

```
3. **Volume kernel structure.**

```cpp
+// HOST SETUP (capture device pointers + reshape views):
+const real_t *Q_data = Q.Read();
+real_t *rhs_data = rhs.ReadWrite();
+const auto B = Reshape(qd.B.Read(), ndof, nqp);
+const auto Jinv = Reshape(qd.Jinv.Read(), 3, 3, nqp, ne);
+const auto dsh = Reshape(qd.dshape_ref.Read(), ndof, 3, nqp);
+const auto wd = Reshape(qd.w_detJ.Read(), nqp, ne);
+const auto Ax_d = Reshape(Ax_packed.Read(), NUM_STATE, NUM_STATE);
+const auto Ay_d = Reshape(Ay_packed.Read(), NUM_STATE, NUM_STATE);
+const auto Az_d = Reshape(Az_packed.Read(), NUM_STATE, NUM_STATE);
+
+mfem::forall(ne, [=] MFEM_HOST_DEVICE (int e) {
- const real_t *Q_e = Q_data + e * ndof; // component-major requires
- // separate index math per c
+ // No raw `Q.GetData()` — use the captured `Q_data` pointer from
+ // `Q.Read()` above. Same for rhs (via `rhs.ReadWrite()`).
 real_t Q_qp[NUM_STATE];
 for (int q = 0; q < nqp; q++) {
 for (int c = 0; c < NUM_STATE; c++) {
 Q_qp[c] = 0.0;
 for (int i = 0; i < ndof; i++) {
 Q_qp[c] += B(i, q) * Q_data[c*ndof_total + e*ndof + i];
 }
 }
 // ...
 for (int c = 0; c < NUM_STATE; c++) {
 for (int i = 0; i < ndof; i++) {
 real_t val = 0.0;
 for (int d = 0; d < 3; d++) { val += G_e iq_d * F[d][c]; }
- rhs[c*ndof_total + e*ndof + i] += w * val;
+ // No atomics needed in volume kernel (each (e,c,i) is
+ // written by exactly one thread — the one handling e).
+ rhs_data[c*ndof_total + e*ndof + i] += w * val;
 }
 }
});
```

Apply analogous fix to the mass-inverse and spatial-derivative pseudocode blocks.

Test case:

```
TEST(R_GPU_008_VolumeKernelReadSync, NoStaleHostData) {
 Q.UseDevice(true);
```

```

// Mutate Q on host through HostWrite(), then call ComputeVolumeRHS
// (device path). Result must reflect the host-mutated values, not
// an earlier device snapshot.
}

```

[R-GPU-009] MODERATE [Phase 4 / face table] — **Array<int>** device readiness not specified; plan uses **UseDevice** terminology that doesn't apply

Category: ASSUMPTION / QUALITY

Description: Phase 4 defines:

```

struct FaceTable {
 Array<int> face_indices;
 Array<int> elem1_indices;
 Array<int> elem2_indices;
 Vector normals;
 // ...
};

```

and Appendix C says these are “UseDevice(true)”. But `mfem::Array<T>` is not a `Vector`; it does not have a `UseDevice(true)` method. Instead, `Array<T>` uses an internal `Memory<T>` whose memory class is set at construction (e.g., via `Array<int>::Array(int size, MemoryType mt)`) or via `array.GetMemory().SetHostPtrOwner(...)`. Access for device kernels must use `array.Read()` / `array.Write()`, which DO exist on `Array<T>` and route through the memory manager.

If the implementer copies the pattern from Phase 3's `Vectors` verbatim, they'll call `face_indices.UseDevice(true)` — compile error.

Trigger: Implementer follows Appendix C's residency table literally.

Expected behavior: Plan must distinguish: - `Vector` → `UseDevice(true)` + `Read()` / `Write()` / `ReadWrite()`. - `Array<T>` → no `UseDevice`; allocate with `Array<int>(n, Device::GetDeviceMemoryType())` OR rely on `Array<T>::Read()` triggering host→device copy on first device use.

Suggested fix:

1. In Phase 4 face-table struct definition:

```

struct FaceTable {
- Array<int> face_indices; // mesh face id per bucket entry
- Array<int> elem1_indices; // adjacent element index
- Array<int> elem2_indices; // -1 for boundary faces
+ // Array<int> uses Memory<int>; allocate with device memory class:
+ // face_indices = Array<int>(n, Device::GetDeviceMemoryType());
+ // and access via face_indices.Read() in kernels. Do NOT call
+ // UseDevice – that method is on Vector, not Array.
+ Array<int> face_indices;
+ Array<int> elem1_indices;
+ Array<int> elem2_indices;
 Vector normals; // n[q,d,f]; UseDevice(true)
}

```

2. In Appendix C:

```

- | `FaceTable::{normals,w_detJf,shape*}` | Vector | WaveOperator | device | yes | 4 |
+ | `FaceTable::{normals,w_detJf,shape*}` | Vector | WaveOperator | device | yes (UseDevice on e
+ | `FaceTable::{face_indices,elem1_indices,elem2_indices,...}` | Array<int> | WaveOperator | devi

```

Test case:

```
TEST(R_GPU_009_FaceTable, ArrayDeviceAccess) {
 FaceTable ft;
 ft.face_indices = Array<int>(10, Device::GetDeviceMemoryType());
 for (int i = 0; i < 10; ++i) ft.face_indices[i] = i;
 const int *d = ft.face_indices.Read(); // host-device copy
 mfem::forall(10, [=] MFEM_HOST_DEVICE (int i) {
 // d[i] must equal i - confirms read path
 });
}
```

---

[R-GPU-010] MODERATE [Phase 4 / shared face] — Plan assumes **ParGridFunction::ExchangeFaceNbrData** uses device buffers under CUDA-aware MPI, but this requires **vdim=NUM\_STATE** ghost GF — not yet wired in plan

Category: EDGE\_CASE / ASSUMPTION

Description: Phase 4 says “With CUDA-aware MPI: ExchangeFaceNbrData posts device→device sends/recvs”. This is true in principle, but only if the ParGridFunction and its underlying ParFiniteElementSpace are constructed with device-resident memory and the MFEM build is linked against a CUDA-aware MPI.

The existing wave operator has ghost\_gf\_ (vdim=1 per wave\_operator.hpp:781–797) and ghost\_gf\_full\_state\_ (vdim=NUM\_STATE per R-1601) — both currently allocated without device residency. The plan does not specify which one is used by the device face kernel, nor does it call out the need to re-allocate these GFs with MemoryType::DEVICE or to mark them UseDevice(true).

Also: per the existing R-1601 contract, the vdim=NUM\_STATE ghost is used by EvaluateBulkAtFaultQP-sCanonical (Phase 5 picks this up) but the vdim=1 ghost is used by ComputeSharedFaceFluxRHS (Phase 4 target). The plan inherits both without auditing whether the vdim=1 ghost can be batched for the kernel; if not, the device kernel needs 9 separate ghost exchanges per call (8× cost vs the batched path R-1601 already established).

Trigger: Phase 4 implementer wires the device kernel against ghost\_gf\_ (vdim=1) per-component-loop, producing 9 MPI rounds per ComputeSharedFaceFluxRHS call.

Expected behavior: Plan must specify: - The device shared-face kernel must use ghost\_gf\_full\_state\_ (vdim=NUM\_STATE batched, R-1601 contract) for the macro-step path. - Or, alternatively, batch the existing ghost\_gf\_ per-component exchange into one MPI\_Alltoall round. - Whichever path: the GF's ParFiniteElementSpace must be constructed with Device::GetDeviceMemoryType() so the FaceNbrData() buffer is device-resident.

Suggested fix (Phase 4 plan body):

```
6. **Shared face flux kernel.** Pre-requisite: ghost data must be
 present in `ghost_gf_>FaceNbrData()` before kernel launch.
+ The kernel SHOULD consume `ghost_gf_full_state_>FaceNbrData()`
+ (vdim=NUM_STATE, byNODES) – the R-1601 batched ghost GF – so a
+ single `ExchangeFaceNbrData` call covers all 9 components in one
+ MPI round. Reading the per-component `ghost_gf_` (vdim=1) from
+ the kernel would require 9 separate exchanges per macro-step,
+ 8× the MPI cost.
+ Phase 4 explicitly relocates the macro-step path to consume
+ `ghost_gf_full_state_`; the legacy vdim=1 path
+ (per R-1504 deep-copy correctness note in wave_operator.hpp:782)
+ stays on host until R-1504 is resolved.
```

- With CUDA-aware MPI: ``ExchangeFaceNbrData`` posts device→device sends/recvs; data is already device-resident.
- Without CUDA-aware MPI: insert a ``device→host`` copy before ``ExchangeFaceNbrData`` and a ``host→device`` copy after. Mark with ``// PERF[G-401]: requires CUDA-aware MPI to avoid this transfer.``
- + - The ``ParFiniteElementSpace`` underlying ``ghost_gf_full_state_`` must be constructed with ``Device::GetDeviceMemoryType()`` (or its ``FaceNbrData()`` buffer must be re-allocated with that memory type at first device-kernel call) so the receive buffer lives in device memory.

Test case:

```
TEST(R_GPU_010_SharedFaceGhost, BatchedExchangeDeviceResident) {
 // np=2 setup, build WaveOperator with --device cuda.
 wave.SomeMethodThatTriggersSharedKernel();
 // ASSERT: MPI exchange count for that call is 1 (batched), not 9.
 // ASSERT: ghost_gf_full_state_→FaceNbrData().UseDevice() == true
}
```

---

[R-GPU-011] MODERATE [Phase 0 / **seas::InitDevice**] — Listed config string “cpu,debug” is non-standard and untested

Category: ASSUMPTION

Description: Phase 0’s Validate rule: > Validate, reject any value not in {“”, “cpu”, “cuda”, “hip”, “debug”, “cpu,debug”}

The string “cpu,debug” is plausible (Device::Configure parses comma-separated backends) but is non-standard. MFEM’s own examples use “debug” alone (the DEBUG\_DEVICE backend already implies a host memory pool). “cpu,debug” would mark BOTH Backend::CPU and Backend::DEBUG\_DEVICE — not the intended “debug variant of cpu”.

If the plan wants to allow combined backends, the validation must explicitly enumerate the legal combinations rather than fix an arbitrary one. Otherwise, drop “cpu,debug” and accept only single-backend strings.

Trigger: TOML with [runtime] device = “cpu,debug” — Validate accepts it, Device::Configure marks two backends, MFEM’s behavior is undefined for this combination.

Expected behavior: Either: - Drop “cpu,debug”; document “debug” as the host-emulated device for CI without GPUs. - Or accept the full set of MFEM backend combinations (don’t try to enumerate; delegate validation to Device::Configure’s MFEM\_VERIFY(it != bmap.end(), ...)).

Suggested fix (Phase 0 plan body):

- - In ``Validate``, reject any value not in ``{“”, “cpu”, “cuda”, “hip”, “debug”, “cpu,debug”}``.
- + - In ``Validate``, reject any value not in the single-backend set ``{“”, “cpu”, “cuda”, “hip”, “debug”}``. Multi-backend strings (e.g., “cuda,debug”) are valid for Device::Configure but the interaction with HYPRE / PETSc is undertested; defer to Phase 7.

Test case:

```
TEST(R_GPU_011_ValidateDevice, RejectsMultiBackendString) {
 SEASConfig cfg;
 cfg.runtime.device = “cuda,debug”;
```

```

 ASSERT_THROW(SEASConfigParser::Validate(cfg), std::runtime_error);
}

```

[R-GPU-012] MODERATE [Phase 6 / shared face table] — Cross-phase reuse of **face\_table\_fault\_interior\_** is asserted but ownership / lifetime not specified

Category: ASSUMPTION

Description: Phase 6 says: > Reuse Phase 4 face table. BP5's fault faces are the same set as the dynamic-rupture interior fault faces (both share FaultBasis). Phase 6 reuses `face_table_fault_interior_` from Phase 4 — at construction, BP5 must call the same `InitFaceTables()` helper.

But `WaveOperator` is the owner of `face_table_fault_interior_` per Phase 4 ("Add private members to `wave_operator.hpp`"). BP5's `ElasticityDomainOperator` is a completely separate class (`domain/elasticity_operator.hpp`) that does not own a `WaveOperator`. The plan's "reuse" implies one of: - BP5 builds its own `face_table_fault_interior_` (duplicate, not shared) — but then the refactor of `InitFaceTables` into a free function is needed. - BP5 receives a pointer to a `WaveOperator`'s face table — but BP5 doesn't have a `WaveOperator`.

Neither is spelled out. The plan's refactor instruction "refactor `InitFaceTables` into a free function in `face_table.hpp`" addresses the first interpretation but doesn't make the ownership shift in Appendix C, which still lists the face tables as owned by `WaveOperator`.

Trigger: Phase 6 implementer follows the literal instruction, finds no `WaveOperator` in BP5's call chain, guesses what to do.

Expected behavior: Plan must explicitly state: - The `FaceTable` struct moves to a shared header `domain_or_dynamic/kernels/face_table.hpp`. - BP5's `ElasticityDomainOperator` owns its OWN `face_table_fault_interior_` member; it is NOT shared with `WaveOperator`. - The `InitFaceTables` builder is a free function taking a `FiniteElementSpace`, a `FaultBasis`, and a list of fault face mesh indices, returning a populated `FaceTable`. - Appendix C lists two separate `FaceTable` owners (`WaveOperator` for TPV, `ElasticityDomainOperator` for BP5).

Suggested fix (Phase 6 plan body):

```

1. **Reuse Phase 4 face table.** BP5's fault faces are the same set as
- the dynamic-rupture interior fault faces (both share
- `FaultBasis`). Phase 6 reuses `face_table_fault_interior_` from
- Phase 4 — at construction, BP5 must call the same
- `InitFaceTables()` helper. Refactor `InitFaceTables` into a free
- function in `face_table.hpp` so both `WaveOperator` (Phase 4) and
- `ElasticityDomainOperator` (Phase 6) can call it.
+ the dynamic-rupture interior fault faces from the *physics-set
+ perspective*, but the two owning classes are separate. Phase 6:
+ - Promotes `FaceTable` (struct) and `InitFaceTables` (free
+ function) from `dynamic/kernels/face_table.hpp` into a shared
+ header `common/kernels/face_table.hpp`.
+ - `ElasticityDomainOperator` gains its own
+ `mutable kernels::FaceTable face_table_fault_` private
+ member, independent from any `WaveOperator` instance.
+ - The free function `InitFaultFaceTable(const FESpace&,
+ const FaultBasis&, const Array<int>& fault_face_indices,
+ FaceTable&)` is called once at first device kernel use, by
+ both `WaveOperator::InitFaceTables` and
+ `ElasticityDomainOperator::InitDeviceBuffers`.

```

Apply matching update to Appendix C (add `ElasticityDomainOperator` row).

Test case:

```
TEST(R_GPU_012_FaceTableShared, BP5AndWaveOpHaveIndependentTables) {
 // Build both a BP5 ElasticityDomainOperator and a WaveOperator on
 // disjoint meshes. Verify each owns its own FaceTable; modifying
 // one does not affect the other.
}
```

---

[R-GPU-013] MODERATE [Phase 3 / mesh-wide assumption] — “Uniform geometry” abort is too strict; mesh may legitimately mix tet orientations or curvature

Category: EDGE\_CASE

Description: Phase 3 says: > Heterogeneous mesh (mixed tet + hex) — Phase 3 requires uniform geometry. Abort if `fes_>GetMesh()->GetElementGeometry(0) != ...` varies across elements.

Reasonable for the tet-only meshes used in TPV/BP5. But “uniform” implicitly means same geometry type AND same DOF count per element. For curvilinear meshes (which Tandem does support; BP5/TPV may add later) the Jacobian varies per element but the geometry type is constant. Phase 3’s `Jinv` packing assumes constant Jacobian per element (computed once per element); curvilinear elements have QP-varying Jacobians.

The plan’s `QuadratureData` allocates `Jinv`:  $ne \times 3 \times 3 \times 8 \text{ B} = 72 \text{ MiB}$  for 1M elements — that’s per-element, not per-QP. Curvilinear elements would need per-QP Jacobians:  $ne \times nqp \times 3 \times 3 \times 8 \text{ B}$ , growing 56× for  $p=4$  tet.

This isn’t a bug today (current meshes are linear), but the plan should flag the assumption explicitly so future curvilinear support doesn’t silently produce wrong physics.

Trigger: Future PR adds a curvilinear-tet mesh; Phase 3 kernel runs on it without abort, producing wrong volume integrals.

Expected behavior: Plan must: - State the linear-tet assumption explicitly. - Add `MFEM_VERIFY(mesh.GetNodes() == nullptr || mesh.GetNodes()->FESpace()->GetOrder() == 1, "curvilinear meshes not supported on device path");` to the abort gate.

Suggested fix (Phase 3 plan body):

```
- **Heterogeneous mesh (mixed tet + hex)** - Phase 3 requires uniform
 geometry. Abort if `fes_>GetMesh()->GetElementGeometry(0) != ...`
 varies across elements.
+ - **Curvilinear mesh (high-order Nodes)** - Phase 3 assumes a
+ per-element-constant Jacobian (linear tets). Abort with
+ `MFEM_VERIFY(mesh.GetNodes() == nullptr || mesh.GetNodes()->
+ FESpace()->GetOrder() == 1, "device path requires straight-sided
+ tets - found curvilinear Nodes");` at `InitDeviceBuffers()`.
+ Curvilinear support requires per-QP Jacobian storage; deferred.
```

Test case:

```
TEST(R_GPU_013_CurvilinearMesh, AbortsOnHighOrderNodes) {
 Mesh mesh = MakeTetMesh(...);
 mesh.SetCurvature(2); // forces high-order Nodes
 ASSERT_DEATH(WaveOperator wave(mesh, /*order=*/2, ...);
 wave.InitDeviceBuffers();,
 "device path requires straight-sided tets");
}
```

[R-GPU-014] MODERATE [Phase 3 / volume kernel] — **Q.UseDevice(true)** is never set for the input vector passed in by the ODESolver

Category: EDGE\_CASE / ASSUMPTION

Description: Phase 3 says Mult does `if (DeviceIsActive()) { dQdt.UseDevice(true); }` at the top — only for the output. But the input Q is owned by the ODESolver (RK4Solver, PetscODESolver, etc.) and may not have been `UseDevice(true)`-marked. If the input Q lives on the host and the kernel calls `Q.Read()`, the memory manager will copy host→device on every step.

For RK4 with 4 stages and one step copy per stage, this is 4 wasted host→device transfers per macro step — for TPV102 200m with ~1M ndof and 9 components,  $\sim 73 \text{ MB} \times 4 = 290 \text{ MB/step} \times 10^4 \text{ steps} = \sim 3 \text{ TB}$  of PCIe traffic over a 60-s run. That alone can negate the GPU speedup.

The fix is to mark the state vector itself (the one the ODESolver passes through `Init`) `UseDevice(true)` once at the driver level, BEFORE the ODE loop starts. Phase 3 doesn't say this.

Trigger: Phase 3 lands as written; user runs `--device cuda`; profile shows PCIe traffic dominates wall clock.

Expected behavior: Plan must require the driver to flip `state.UseDevice(true)` on the ODE state vector ONCE, before `ode_solver→Init(...)`.

Suggested fix (Phase 3 plan body, drivers section):

```
- **`Mult`** ([line 684]...):
 - Insert if (DeviceIsActive()) { dQdt.UseDevice(true); } at the top.
+ - The input `Q` is owned by the driver / ODESolver. Document a
+ DRIVER CONTRACT in `tpv102_driver.cpp` / `tpv104_driver.cpp` /
+ `tpv205_driver.cpp`: the ODE state `Q` (the `Vector` passed
+ into `ode_solver→Init(...)`) MUST be `UseDevice(true)` BEFORE
+ the time loop starts, so per-step `Mult` does not trigger
+ host→device copies of the full state.
+ - Add `Vector::UseDevice(true)` call sites in the three TPV
+ driver setup sections, gated on `Device::Allows(DEVICE_MASK)`.
```

Add to the Phase 3 driver acceptance criteria:

```
+ [] TPV102 1-s run with `--device cuda` shows < 5% of wall time in
+ host→device transfers (via nvprof --csv `dtoh`/`htod` columns).
```

Test case:

```
TEST(R_GPU_014_StateResidency, ODEStateUseDeviceSet) {
 tpv102_driver::Setup setup(/*device=*/"cuda");
 ASSERT_TRUE(setup.GetState().UseDevice());
}
```

---

[R-GPU-015] LOW [Phase 0 / Makefile] — Plan delegates “guard for future phases” but does not specify what the guard is

Category: QUALITY

Description: Phase 0 Makefile bullet says: > Add `nvcc / hipcc` host-compiler flags only when actually building kernel `.cu / .hip.cpp` files in later phases; Phase 0 introduces no `.cu` files yet so this is a guard for future phases.

This is too vague. “A guard for future phases” is not actionable. The implementer doesn't know whether to add a placeholder `ifeq ($(MFEM_USE_CUDA), YES)` block now (and leave the kernel-rule body empty) or to skip the Makefile change entirely and revisit in Phase 3.



Trigger: Phase 0 lands with no Makefile change; Phase 3 implementer has to re-derive the build flags from scratch and may produce inconsistent flag ordering.

Expected behavior: Plan must say one of: - “Phase 0 adds a placeholder block `ifeq ($(MFEM_USE_CUDA), YES) $(eval ...) endif` that is empty; Phase 3 fills it with `.cu` pattern rules.” - “Phase 0 does not touch the Makefile; Phase 3 owns the Makefile changes.”

Suggested fix (Phase 0 plan body):

```
- - `miniapps/seas/Makefile` - add a `MFEM_USE_CUDA ?=` and
- `MFEM_USE_HIP ?=` detection block reading the value from
- `$(CONFIG_MK)`. Emit a clear error if `MFEM_USE_CUDA=YES` and
- HYPRE was built without GPU support (detect by grepping
- `HYPRE_LIB`). Add `nvcc` / `hipcc` host-compiler flags only when
- actually building kernel `.cu` / `.hip.cpp` files in later phases;
- **Phase 0 introduces no `.cu` files yet** so this is a guard for
- future phases.
+ - `miniapps/seas/Makefile` - add a `MFEM_USE_CUDA ?=` and
+ `MFEM_USE_HIP ?=` detection block reading the value from
+ `$(CONFIG_MK)`. Emit a clear error if `MFEM_USE_CUDA=YES` and
+ HYPRE was built without GPU support (detect by grepping
+ `HYPRE_OPT` for `HYPRE_USING_GPU`). Phase 0 introduces no kernel
+ files (no `.cu` / `.hip.cpp`), so the Makefile change is limited
+ to (a) reading the new flags, (b) emitting build-time
+ compatibility checks, and (c) adding an empty placeholder
+ `SEAS_KERNEL_OBJS :=` variable that Phase 3 will populate.
```

Test case: Build-system smoke test (Phase 0 acceptance already covers).

---

[R-GPU-016] LOW [Phase 5 / PML kernel] — Inline PML loop pseudo-code missing in Phase 5

Category: QUALITY

Description: Phase 5 says: > The PML inline branch ([line 4763]...) runs a separate per-element host loop. Mirror it as kernels::ApplyPMLDampingADERDevice(I, dt, ...) and dispatch.

But Phase 4 already covered ApplyPMLDamping (Mult path). Phase 5 duplicates the kernel for the ADER path. The two kernels differ only in: - Input vector (I instead of Q). - Background subtraction (dt \* bulk\_bg\_ instead of bulk\_bg\_).

These can share 90% of the code via a parameterized common helper. The plan does not call out the sharing, risking duplicate (and divergent) implementations.

Trigger: Phase 4 and Phase 5 PML kernel implementations drift apart, producing bugs only in long ADER + PML runs.

Expected behavior: Plan must say: implement one ApplyPMLDampingDeviceImpl(input, dt\_or\_unit, ...) template/free function; both Phase 4 (ApplyPMLDamping) and Phase 5 (ApplyPMLDampingADERDevice) thin-wrap it.

Suggested fix (Phase 5 plan body):

```
- - The PML inline branch ([line 4763]...) runs a separate per-element
- host loop. Mirror it as `kernels::ApplyPMLDampingADERDevice(I, dt, ...)`
- and dispatch.
+ - The PML inline branch ([line 4763]...) runs a separate per-element
+ host loop. Implement ONCE as a shared free function
+ `kernels::ApplyPMLDampingDeviceImpl(const Vector &input, real_t
+ bg_scale, /* other args */, Vector &rhs)`. Phase 4's
```

```
+ `ApplyPMLDamping` thin-wraps with `bg_scale = 1.0`; Phase 5's
+ `ApplyPMLDampingADERDevice` thin-wraps with `bg_scale = dt`.
+ - Move `kernels::ApplyPMLDampingDeviceImpl` to
+ `dynamic/kernels/pml_kernels.hpp` (new in Phase 4).
```

Test case:

```
TEST(R_GPU_016_PMLSharedImpl, ADERAndMultPathsCallSameImpl) {
 // Static-assert that there is exactly one definition of
 // ApplyPMLDampingDeviceImpl by grepping the symbol table.
}
```

---

[R-GPU-017] LOW [Phase 7 / CI] — Plan assumes GitHub Actions but does not check whether the repo uses it

Category: ASSUMPTION

Description: Phase 7 says: > .github/workflows/seas\_gpu\_ci.yml (NEW; if CI is GitHub Actions)

Conditional “if CI is GitHub Actions” — but the plan doesn’t check whether the repo actually uses GitHub Actions (it might use GitLab CI, Buildkite, internal Jenkins, etc.). Without that, the implementer might write a useless file.

A quick scan would resolve this:

```
ls .github/workflows/ # GitHub Actions
ls .gitlab-ci.yml # GitLab
ls .buildkite/ # Buildkite
```

Trigger: Implementer creates .github/workflows/seas\_gpu\_ci.yml on a repo that doesn’t have GitHub Actions wired up.

Expected behavior: Plan should ask the implementer to first check what CI is in use, then add the appropriate config file. Or, simpler: just provide the CI matrix as documentation in gpu\_dev/CI\_MATRIX.md and let the maintainer wire it into whatever CI they have.

Suggested fix (Phase 7 plan body):

```
- `.github/workflows/seas_gpu_ci.yml` (NEW; if CI is GitHub Actions) -
- CPU-only smoke + `Backend::DEBUG_DEVICE` equivalence run on every PR.
+ `miniapps/seas/document/gpu_dev/CI_MATRIX.md` (NEW) - documents the
+ required CI build matrix (CPU-only smoke, `--device debug`
+ equivalence, optional GPU runner for `--device cuda`).
+ Implementer wires this into the repo's existing CI system
+ (GitHub Actions, GitLab CI, etc.) - investigate which one is in
+ use first via `ls .github/ .gitlab-ci.yml .buildkite/` before
+ writing the YAML.
```

Test case: N/A (documentation-only).

---

[R-GPU-018] LOW [Convention constraints] — “no **std::vector**” is too strict for kernel SETUP code

Category: QUALITY

Description: Convention constraints say: > Avoid **std::vector**, **std::map**, **std::sort**, virtual dispatch, exceptions.

True INSIDE a forall kernel body. But the surrounding setup (InitFaceTables, BuildQuadratureData) is host code and routinely uses `std::vector` as scratch storage. The blanket “avoid” wording is too strong for setup code; it should be scoped to kernel bodies only.

Trigger: Implementer reads “avoid `std::vector`” and reinvents `std::vector` on the host to satisfy the rule, wasting effort and introducing home-grown bugs.

Suggested fix:

```
- Every `mfem::forall` kernel must:
 - ...
- - Avoid `std::vector`, `std::map`, `std::sort`, virtual dispatch, exceptions.
+ - INSIDE the kernel body (the lambda passed to `mfem::forall`):
+ no `std::vector`, `std::map`, `std::sort`, virtual dispatch,
+ exceptions. Setup code on the host may use these freely.
```

Test case: N/A (style guidance).

---

## Summary

- Critical issues: 5 (R-GPU-001, R-GPU-002, R-GPU-003, R-GPU-004, R-GPU-005)
- Moderate issues: 9 (R-GPU-006 through R-GPU-014)
- Low issues: 4 (R-GPU-015, R-GPU-016, R-GPU-017, R-GPU-018)
- Plan compliance: N/A (no implementation yet; this is plan-document review)
- Verdict: FAIL — must fix before proceeding to implementation

The 5 CRITICAL findings would all produce wrong code, runtime aborts, or untraceable wrong physics if the implementing agent followed the plan literally:

1. R-GPU-001 (virtual Rate on device) — guaranteed kernel crash.
2. R-GPU-002 (MPIContext insertion point) — guaranteed compile error or wrong init in 3 of 4 drivers.
3. R-GPU-003 (MAX\_DOF=84 for “p=4 tet”) — guaranteed buffer over-/under-sizing and risk of UB if implementer uses MAX\_DOF as loop bound.
4. R-GPU-004 (AtomicMax) — guaranteed compile failure on option B.
5. R-GPU-005 (raw `atomicAdd`) — guaranteed HIP build break / non-portable kernel.

The MODERATE findings would either produce wrong test outcomes, performance pitfalls (~10× slower than achievable), or ambiguities that force implementer to guess.

## Unreviewed Areas

- Numerical correctness of the math snippets (Cauchy-Kovalevskaya recursion, Godunov flux, friction equations) — these were taken from the existing CPU code which is already verified; not re-derived here.
- Tandem comparison harness for Phase 7 — referenced but not inspected.
- PETSc TS GPU integration depth — Phase 1 has a startup gate but the actual `MuLt` callback signature contract under PETSc CUDA is not audited (would require PETSc source).
- dFEM alternative path (mentioned as open question in the analysis doc) — out of scope of this review.